Laboratório Prolog · Aula 13

Gabriel Arantes · RA 24169751-2 · Turma ESOFT6S-B
Gerado em 20/10/2026, 20:19:52 · tempo registrado no cronômetro: 53:37

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) deve dar L = scheme (uma resposta). paradigma(java, logico) deve dar false, porque esse fato não está na base.

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
--- experimentos (linhas do quadro ativadas) ---
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) tem a variável P: pergunta "qual é o paradigma do prolog?" e o Prolog responde P = logico. paradigma(prolog, logico) não tem variável: pergunta "prolog é do paradigma logico?" e só pode responder true ou false (aqui, true).

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que o programa não sabe isso. false significa só que a meta não pode ser provada com os fatos da base (mundo fechado). A base tem paradigma(java, orientado_a_objetos) e nada sobre logico, então não dá para concluir nada sobre o Java real.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas (lucia, marta, tereza, joana, bia, ana, dora). (mae(M, F), mulher(F)) terá 4 pares: lucia-tereza, marta-joana, tereza-bia e joana-ana.

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
--- experimentos (linhas do quadro ativadas) ---
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mae(A,B),homem(B).
   mae(lucia,vern),homem(vern)
   mae(marta,paulo),homem(paulo)
   mae(joana,jake),homem(jake)
   mae(dora,leo),homem(leo)
?- homem(A),mulher(A).
   false.
?- mae(joana,jake),mae(joana,ana).
   mae(joana,jake),mae(joana,ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Significa: "quais são as mães (Mae) e as filhas (Filha) tais que Mae é mãe de Filha e Filha é mulher?". Teve 4 respostas: (lucia, tereza), (marta, joana), (tereza, bia) e (joana, ana).

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

A hipótese do mundo fechado: tudo que não pode ser provado pela base é considerado falso. Em situações reais isso é um problema porque falta de informação não é o mesmo que falsidade: se o cadastro está incompleto, o Prolog responde false ("não") quando o certo seria "não sei", e podem sair conclusões erradas.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Apareceram na ordem lucia, marta, tereza, joana, bia, ana, dora. É a ordem em que os fatos foram escritos no arquivo: o Prolog percorre a base de cima para baixo, em busca em profundidade.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) deve dar X = paulo (irmão de joana, mãe de jake). primo(leo, X) deve dar jake e ana (filhos de joana, irmã de paulo, pai de leo).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Vão falhar: f(X, X) = f(a, b) (X não pode ser a e b ao mesmo tempo), [A|B] = [] (lista vazia não tem cabeça) e 2 + 3 = 5 (2+3 é uma estrutura, não o número 5). f(X, X) = f(a, Y) unifica, com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
##### regras.pl
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
```

```
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
--- experimentos (linhas do quadro ativadas) ---
?- irmao(A,B).
   irmao(vern,tereza)
   irmao(tereza,vern)
   irmao(joana,paulo)
   irmao(paulo,joana)
   irmao(jake,ana)
   irmao(ana,jake)
?- tio(A,B).
   tio(vern,bia)
   tio(paulo,jake)
   tio(paulo,ana)
?- ancestral(A,leo).
   ancestral(dora,leo)
   ancestral(paulo,leo)
   ancestral(lucia,leo)
   ancestral(marta,leo)
   ancestral(osvaldo,leo)
   ancestral(vern,leo)
?- primo(A,B).
   primo(bia,joana)
   primo(bia,paulo)
   primo(jake,leo)
   primo(ana,leo)
   primo(joana,bia)
   primo(paulo,bia)
   primo(leo,jake)
   primo(leo,ana)
[vern,tereza,joana,paulo,jake,ana,leo,bia]

##### unificacao.pl
--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3  =  5    NAO unifica
```

```
--- experimentos (linhas do quadro ativadas) ---
f(A,B)  =  f(1,2)    unifica   A = 1   B = 2
g(A,b)  =  g(a,A)    NAO unifica
[A,B|C]  =  [1,2,3,4,5]    unifica   A = 1   B = 2   C = [3,4,5]
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque uma recursão precisa de dois casos. Caso base: ancestral(X, Y) :- progenitor(X, Y) (pai ou mãe direto). Caso recursivo: ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y) (X é progenitor de alguém que é ancestral de Y). Só com o caso base acharia apenas pais e mães diretos; só com o recursivo nunca teria onde parar.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, irmao(jake, X) passa a devolver também irmao(jake, jake), além de irmao(jake, ana). Jake tem a mesma mãe e o mesmo pai que ele mesmo, então a regra se satisfaz com X = Y. A meta X \== Y é necessária para excluir esse caso: ninguém é irmão de si mesmo.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X, X) = f(a, b), a mesma variável X teria que valer a e b ao mesmo tempo, o que é impossível: não unifica. Em f(X, X) = f(a, Y), X = a e, como o segundo argumento também é X, Y precisa ser igual a X: unifica com X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não. Uma função de Java ou Scheme tem entradas e saída fixas: os parâmetros entram e o resultado sai, em uma única direção. avo é uma relação: qualquer argumento pode ser variável. avo(X, jake) acha os avós (marta, vern) e avo(osvaldo, N) acha os netos (joana, paulo, bia). O mesmo código serve para todas as direções por causa da unificação e do backtracking.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X, Y, [1, 2, 3]) vai dar 4 respostas: X = [] e Y = [1,2,3]; X = [1] e Y = [2,3]; X = [1,2] e Y = [3]; X = [1,2,3] e Y = [].

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
```

```
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
--- experimentos (linhas do quadro ativadas) ---
?- concatena(A,[b,c],[a,b,c]).
   concatena([a],[b,c],[a,b,c])
?- inverte([a,b,c,d],A).
   inverte([a,b,c,d],[d,c,b,a])
?- tamanho([x,y],A).
   tamanho([x,y],2)
?- membro(A,[]).
   false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([], 0). Passo recursivo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1. Em tamanho([a,b,c], N), cada chamada deixa pendente o cálculo N is M + 1 enquanto espera o tamanho da cauda: [a,b,c] espera [b,c], que espera [c], que espera []. Ao chegar em [] (0), o resultado volta somando 1 em cada nível: 1, 2 e 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque a lista [1,2,3] pode ser cortada em 4 pontos (prefixo com 0, 1, 2 ou 3 elementos) e o Prolog encontra cada um por backtracking. O anexar do Scheme é uma função: só recebe duas listas e devolve a junção. Ele não consegue separar uma lista em duas, nem descobrir qual lista falta para formar um resultado.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Unificação com backtracking. A 1ª cláusula unifica X com a cabeça (a). Quando se pede outra resposta, o Prolog volta, usa a 2ª cláusula (membro na cauda) e unifica X com b, depois com c. Cada alternativa pendente vira uma nova resposta, sem laço.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 deixa X como a estrutura 2+3 (não calcula). X is 2 + 3 calcula e dá X = 5. fatorial(N, 120) dá erro de instanciação (instantiation_error), porque N é uma variável livre em N > 0.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
--- experimentos (linhas do quadro ativadas) ---
?- A is 5*4+2.
   22 is 5*4+2
?- A=5*4+2.
   5*4+2=5*4+2
?- fatorial(10,A).
   fatorial(10,3628800)
?- fib(20,A).
   fib(20,6765)
?- fatorial(A,120).
   ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica sem calcular: X = 2 + 3 deixa X = 2+3 e 2 + 3 = 5 falha. is avalia o lado direito (que precisa estar todo conhecido) e unifica o resultado com o esquerdo: X is 2 + 3 dá X = 5. =:= compara o valor numérico dos dois lados, sem ligar variáveis: 5 =:= 2 + 3 é verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Em fatorial(N, 120), a regra faz N > 0 e N1 is N - 1 com N sem valor, e a aritmética exige valores conhecidos, por isso dá instantiation_error. concatena só usa unificação, que funciona em qualquer direção. A aritmética (is, >, =:=) não é relacional: só calcula em um sentido.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: mesma estrutura recursiva, com caso base (fatorial de 0 é 1) e passo n * fatorial(n-1). Diferente: em Haskell fact é uma função que devolve o valor como expressão; em Prolog é uma relação com um argumento extra para o resultado (F), que usa N > 0 como teste e is para calcular, com variáveis auxiliares (N1, F1), e só funciona com N conhecido.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dá false e solteiro1(X) dá respostas. maximo2(5, 3, M) dá duas respostas (5 e 3). maximo3(5, 3, 3) dá true (o que é errado): o corte não protege esse uso.

Saída de controle.pl e mensagem de ancestral_ruim

```
##### controle.pl
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   maximo3(5,3,3)
--- experimentos (linhas do quadro ativadas) ---
?- maximo3(8,3,A).
   maximo3(8,3,8)
?- solteiro1(A).
   solteiro1(osvaldo)
```

```
   solteiro1(paulo)
   solteiro1(jake)
?- maximo2(8,3,A).
   maximo2(8,3,8)
   maximo2(8,3,3)

##### laco_infinito.pl
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variable
used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 colorações. Com 2 cores nenhuma (false), porque PR, SP e MS fazem fronteira entre si e precisam de 3 cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog tenta amigo(X) e amigo(Y) em ordem. Em "tenta ana e ana", a meta X \== Y falhou (é a mesma pessoa) e ele voltou ao último ponto de escolha, amigo(Y), tentando bia. Em ana e bia, gosta(ana, cafe) mas gosta(bia, cha): a última meta falhou e ele voltou de novo, tentando caio. Em ana e caio os dois gostam de cafe e saiu a 1ª resposta. O mesmo processo gerou caio e ana. Depois o fail força a busca das demais combinações.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro2, \+ casado(X) vem com X ainda sem valor: casado(X) é provável (X = vern), então a negação falha logo e homem(X) nunca é tentado, dando false. Em solteiro1, homem(X) gera um X primeiro e só depois testa \+ casado(X). Ele diz que osvaldo é solteiro porque não há o fato casado(osvaldo): negação como falha e mundo fechado. Não provar "casado" vira "solteiro", mesmo ele sendo pai de vern.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

Em maximo3(5, 3, 3), a cabeça da 1ª cláusula, maximo3(X, Y, X), não unifica (o 3º argumento é 3, mas X = 5). Ela falha antes de executar o corte, que nunca roda, então a 2ª cláusula maximo3(_, Y, Y)

é usada e responde true. Correção: maximo3(X, Y, Z) :- X >= Y, !, Z = X. e maximo3(_, Y, Y). Assim a unificação do resultado vem depois do corte e maximo3(5, 3, 3) dá false.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

Porque a ordem das metas importa no Prolog. ancestral_ruim chama a si mesma primeiro, com os mesmos argumentos e sem consumir nenhum progenitor (recursão à esquerda), então entra em laço infinito até estourar a pilha (local stack overflow). ancestral_bom consome um progenitor antes de recursar, então a busca avança e termina. Logicamente são equivalentes, mas o Prolog não é puramente declarativo: a ordem das cláusulas e metas controla a busca.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

O programa encontrou 24 colorações com 3 cores, 0 com 2 cores (false, pois PR, SP e MS são vizinhos entre si e precisam de 3 cores) e 1728 com 4 cores. É declarativo porque só descreve o que é uma coloração válida (as cores escolhidas e as fronteiras com cores diferentes), sem laço nem algoritmo de busca: quem procura é o backtracking do Prolog.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, anexar é uma função: recebe duas listas e devolve a junção, em uma só direção. No lógico, concatena é uma relação: o mesmo código junta, separa (concatena(X, Y, [1,2,3]) dá 4 respostas) e testa, pois a prova é feita por unificação e backtracking, sem entrada e saída fixas.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) Controle da ordem de resolução: em laco_infinito.pl, ancestral_ruim diz logicamente o mesmo que ancestral_bom, mas estoura a pilha por causa da ordem das metas (e o corte em maximo3 também mostrou que controlar a busca pode mudar o sentido). 2) Negação como falha e mundo fechado: em controle.pl, solteiro1 afirma que osvaldo é solteiro só porque não há casado(osvaldo), e solteiro2, com as metas em outra ordem, dá false.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras, relações e busca: bancos de dados dedutivos, sistemas especialistas, linguagem natural e restrições/quebra-cabeças (como o mapa e o SEND+MORE), em que basta descrever o que é verdade. Não escolheria para cálculo numérico pesado, interfaces, entrada e saída ou sistemas grandes com desempenho previsível: a aritmética não é relacional, o controle da busca é difícil e Java é mais adequado.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
```

```
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)
```

E1 Regras de família: código escrito

```
irma(X, Y) :- mulher(X), irmao(X, Y).
tia(X, Y) :- progenitor(P, Y), irma(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H * H.
pares([], []).
pares([H|T], [H|R]) :- H mod 2 =:= 0, pares(T, R).
pares([H|T], R) :- H mod 2 =\= 0, pares(T, R).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A).
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).
potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Vis) :- liga(A, C), \+ membro(C, Vis), caminho(C, B, [C|Vis]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :- findall(S, funcionario(_, D, S), L), soma(L, Total).
```